UML Deployment Diagrams

System Architecture & Physical Design

System Analysis & Design - UML Diagrams

What is a Deployment Diagram?

- Structural UML diagram showing physical architecture
- Depicts hardware nodes and software artifacts deployment
- Shows how software components are distributed across hardware
- Visualizes system's runtime processing nodes and connections
- Bridge between software design and physical implementation

Purpose & Importance

- Visualize system topology and hardware infrastructure
- Plan and document deployment strategy for stakeholders
- Identify performance bottlenecks and scalability issues
- Guide DevOps teams in deployment and configuration
- Support capacity planning and resource allocation decisions

Key Components of Deployment Diagrams

Nodes

Physical or virtual devices (servers, workstations, devices)

Artifacts

Software components, files, executables, databases

Communication Paths

Network connections between nodes

Nodes - Execution Environments

Node Type	Description	Examples
Device Node	Physical hardware	Server, PC, Mobile, Printer
Execution Environment	Software container	JVM, Web Server, App Container
Cloud Node	Virtual infrastructure	AWS EC2, Azure VM, GCP Instance

Artifacts - Deployable Components

- Executable files (.exe, .jar, .war, .apk)
- Libraries and dependencies (.dll, .so, .lib)
- Configuration files (web.config, application.yml)
- Database schemas and scripts (.sql)
- Static resources (HTML, CSS, images, documents)

Deployment Diagram Notation

Node (3D Cube)

Artifact (Rectangle)

Communication Path

(Solid line with protocol label)

Types of Deployment Diagrams

1

Specification-Level

Shows types of nodes and artifacts without specific instances

2

Instance-Level

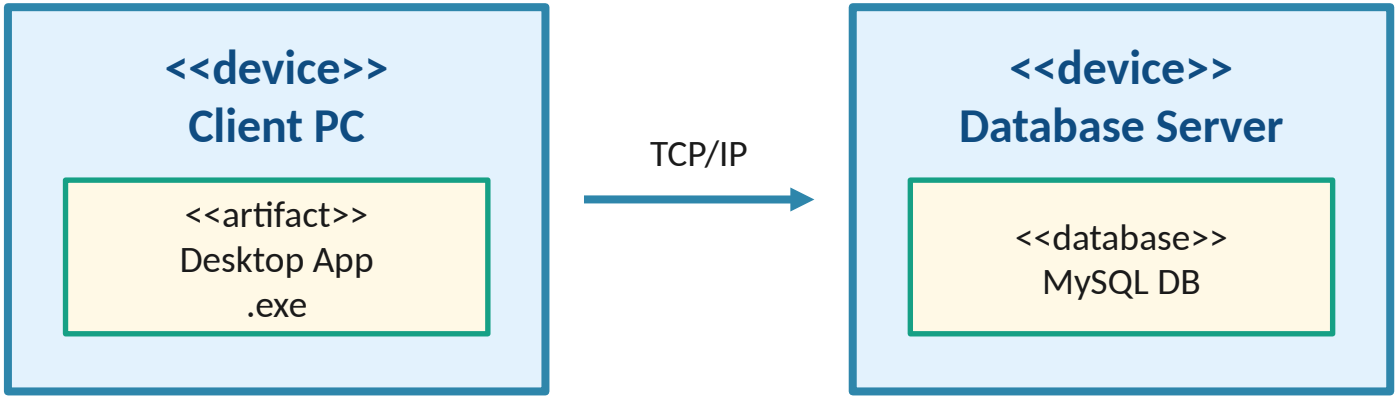
Shows specific instances with actual server names and IPs

Used at different phases: Specification (Design) vs Instance (Production)

Deployment Architecture Categories

Category	Description	Use Case
Single-Tier	All components on one machine	Standalone desktop apps
Two-Tier	Client-Server architecture	Desktop app + Database
Three-Tier	Presentation, Business, Data layers	Web applications
N-Tier	Multiple distributed layers	Enterprise systems
Cloud-Based	Distributed across cloud services	Modern SaaS applications

Two-Tier Architecture Example



Example: Library Management System with desktop client

Three-Tier Architecture Example

- Presentation Layer: User interface in browser
- Business Logic Layer: Application server processes requests
- Data Layer: Database server stores and retrieves data

Real-Life Example: E-Commerce Platform

Deployment: Daraz / Amazon-style Online Shopping

Layer	Node	Artifacts
Client	Web Browser / Mobile App	HTML, React.js, Android APK
Web Tier	Nginx Load Balancer	nginx.conf
App Tier	Node.js App Server (x3)	app.js, API endpoints
Cache	Redis Server	In-memory cache
Database	MySQL Cluster	Product DB, User DB, Orders DB
Storage	AWS S3 Bucket	Product images, documents

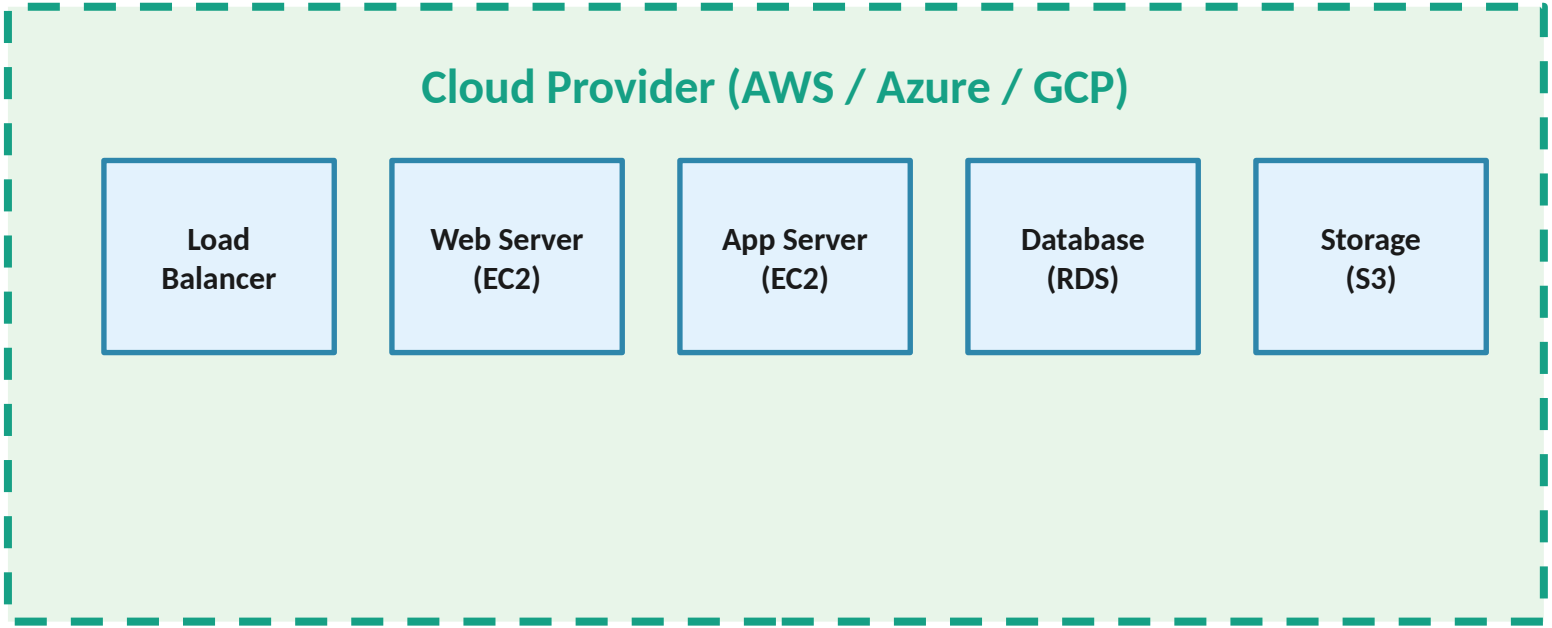
Real-Life Example: Online Banking System

- ATM Machines → Connected to Central Server via VPN
- Mobile Banking App → Deployed on iOS/Android devices
- Web Portal → Hosted on Apache Tomcat Server
- Application Server → Java EE on IBM WebSphere
- Core Banking Database → Oracle RAC Cluster
- Firewall & Security Gateway → DMZ Network

Real-Life Example: Video Streaming (Netflix)

- Client Devices: Smart TV, Mobile, Browser (React/Native apps)
- CDN (Content Delivery Network): Edge servers worldwide
- API Gateway: AWS API Gateway for request routing
- Microservices: Docker containers on Kubernetes clusters
- Databases: Cassandra (NoSQL) for user data & viewing history
- Video Storage: AWS S3 buckets for encoded video files

Cloud-Based Deployment Architecture



Benefits: Scalability, High Availability, Pay-as-you-go, Auto-scaling

Microservices Deployment Pattern

- Each microservice deployed in separate container (Docker)
- Container orchestration using Kubernetes or Docker Swarm
- API Gateway routes requests to appropriate microservice
- Service discovery for dynamic service location
- Independent scaling and deployment of each service

Benefits of Deployment Diagrams

Clear Communication	Stakeholders understand system infrastructure
Deployment Planning	Guide for DevOps and IT operations teams
Performance Analysis	Identify bottlenecks and optimize resources
Scalability Design	Plan for future growth and load distribution
Cost Estimation	Calculate hardware and cloud infrastructure costs

Best Practices for Deployment Diagrams

- Keep diagrams simple - avoid unnecessary complexity
- Use stereotypes (<<device>>, <<artifact>>, <<server>>)
- Label communication paths with protocols (HTTP, HTTPS, JDBC)
- Show both specification and instance-level diagrams
- Update diagrams as architecture evolves
- Include hardware specifications for critical nodes

Tools for Creating Deployment Diagrams

Tool	Type	Key Features
Visual Paradigm	Desktop	Professional UML tool, code generation
Lucidchart	Cloud-based	Collaborative, easy to use, templates
Draw.io (diagrams.net)	Free/Open	Free, browser-based, offline mode
Enterprise Architect	Desktop	Enterprise-grade, full SDLC support
StarUML	Desktop	Lightweight, affordable, cross-platform
PlantUML	Text-based	Code-as-diagram, version control friendly

Key Takeaways

- Deployment diagrams show physical system architecture
- Key elements: Nodes, Artifacts, Communication Paths
- Types: Specification-level vs Instance-level diagrams
- Categories: Single-tier, Two-tier, Three-tier, N-tier, Cloud
- Essential for deployment planning and system optimization
- Used across all modern architectures: Web, Cloud, Microservices